Design Documentation

Interactive House

GitHub: Interactive House — see `SG1_Server_DB/` for server & database code

Revision History

Date	Version	Description	Author
01/03/2026	1.0	Initial design item descriptions focusing on requirements document	Katerina Arvay, Dario Ostojic, Andre Sandblom
17/03/2026	1.1	Added clarification of the current hybrid architecture and outlined planned future changes to the system design	Katerina Arvay, Dario Ostojic, Andre Sandblom
23/04/2026	1.2	Revised design architecture with backup database.	Katerina Arvay, Dario Ostojic, Andre Sandblom
08/05/2026	1.3	D1 updated (password update, LCD), D2 (two-collection Firestore), D3 (two-path architecture), D4 (orange light PWM), D5 (API endpoints) added, traceability added	Katerina Arvay, Dario Ostojic, Andre Sandblom

Color Code Legend

Essential priority	Desirable priority

Design Item List

Design Item	Priority
D1. Use case diagram	Essential
D2. Entity and class diagram	Essential
D3. Interaction sequence diagram	Essential
D4. State diagram	Essential

Design Item Descriptions

D1. Use case diagram

The user can perform several key actions: registering a new account, logging in to authenticate themselves, updating their password, and logging out to terminate their session.

Once authenticated, the user can control various devices in the house. These include opening or closing the door and window, turning lights on or off (white light toggle; orange light PWM brightness), activating or deactivating the fans (Fan INA and Fan INB independently), controlling the buzzer, and setting the LCD text display. Each of these actions corresponds to a specific use case in the system.

D2. Entity and class diagram

The User entity includes two attributes: Username, which uniquely identifies the user, and HashedPassword, which stores the user's encrypted password for secure authentication.

The Device entity represents smart home devices and contains four attributes: Name to identify the device, State to indicate its status (such as on/off or open/closed), Value to hold any additional relevant data and Pin which corresponds to the hardware control pin.

The system stores and manages entities across two Firestore collections. The users/{username} document (managed by the SG1 API) stores the hashed password and per-user device state. The devices/arduino document (managed by the SG3 gateway) stores confirmed physical device states, sensor telemetry (gas, steam, motion), and sync metadata. These two collections serve different client paths and are architecturally independent.

D3. Interaction sequence diagram

The sequence diagram illustrates how users interact with the Smart House system through web or mobile applications via the backend server.

When a new user registers, the client application sends user credentials to the backend API. The server validates the input, securely processes the password, and stores the user information in the system's data storage.

When a user logs in, the client sends authentication credentials to the backend. The server verifies the credentials and, upon success, issues an authentication token that must be included in subsequent requests.

For device control actions (such as turning lights on/off, opening/closing doors or windows, or activating devices), the client sends a request to the backend API along with the authentication token. The backend validates the request, updates the system state, and stores the updated device state in the primary data storage.

The SG3 gateway watches the devices/arduino Firestore document directly (not through the SG1 REST API). When desired state changes, the gateway dispatches serial commands to the Arduino (D:1/D:0 door, N:1/N:0 window, B:1/B:0 buzzer, W white light, O orange light, X/Y fans, M[text]| LCD). The Arduino confirms physical state over serial; the gateway writes confirmed state back to Firestore. SG2 clients observe this document in real time via Firestore onSnapshot listeners.

Logout is handled client-side by deleting the stored JWT token. The server relies on token expiry (2 hours); no server-side revocation is currently implemented.

D4. State diagram

Devices such as white light, fans (INA and INB) and the buzzer follow the same On/Off state machine. The door and window follow an Open/Closed state machine. The orange light uses a continuous PWM value (0–255) via the orange_light.value field rather than binary on/off.

D5. API Endpoint Reference

Traceability: Register user, Authenticate user, Terminate session, all device control requirements, Update password, Delete device field, LCD display. Tests: verify each endpoint returns the correct response for valid and invalid requests.

All endpoints are hosted at: https://us-central1-smarthouse-617fe.cloudfunctions.net/api

Method	Path	Auth	Description
POST	/users/register	x-api-key	Create new user. Provisions 7 default devices.
POST	/users/login	None	Authenticate user. Returns JWT (2h).
PATCH	/users/update-password	Bearer JWT	Update password.
PATCH	/users/device	Bearer JWT	Update device state.
DELETE	/users/device	Bearer JWT	Remove device field.
GET	/users/me	Bearer JWT	Get all device states.
GET	/	None	API health check.

Traceability Overview

The table below cross-references design items with the requirements they address and the tests that verify them.

Design Item	Requirements covered	Tests
Use case diagram	Register user, Login, Logout, Open/close door, Turn lights on/off, Open/close window, Turn fan on/off, Turn buzzer on/off, Retrieve device status, Update password, Delete device field, LCD display	All device and authentication tests
Entity and class diagram	Register user, Manage authorized devices	Verify user is created with hashed password and all default device fields
Interaction sequence diagram	Register user, Login, Logout, all device control, Update password	Registration flow; login and token handling; device command forwarding through gateway
State diagram	Open/close door, Turn lights on/off, Open/close window, Turn fan on/off, Turn buzzer on/off	Verify each device transitions correctly between its allowed states
API endpoint reference	Register user, Login, Logout, Update password, Delete device field, LCD display, Retrieve device status	Verify each endpoint returns the correct response for valid and invalid requests